

МОСКОВСКИЙ ГОСУДАРСТВЕННЫЙ ТЕХНИЧЕСКИЙ УНИВЕРСИТЕТ
им. Н.Э. Баумана

Факультет “Информатика и системы управления”
Кафедра “Системы обработки информации и управления”



Дисциплина “Парадигмы и конструкции языков программирования”

Отчет по рубежному контролю №2

Выполнил:
Студент группы ИУ5-34Б
Шерстеникин Н.С.
Преподаватель:
Гапанюк Ю.Е.

Москва 2025

```

class Stroki:
    def __init__(self, id, content, CharAmount, TableId):
        self.id = id
        self.content = content
        self.CharAmount = CharAmount
        self.TableId = TableId

    def __repr__(self):
        return f"Stroki(id={self.id}, content='{self.content}',
chars={self.CharAmount}, table_id={self.TableId})"

class Table:
    def __init__(self, id, name):
        self.id = id
        self.name = name

    def __repr__(self):
        return f"Table(id={self.id}, name='{self.name}')"

class TableString:
    def __init__(self, id, StringId, TableId):
        self.id = id
        self.StringId = StringId
        self.TableId = TableId

    def __repr__(self):
        return f"TableString(id={self.id}, string_id={self.StringId},
table_id={self.TableId})"

StrokiList = [
    Stroki(1, "бла-бла", 7, 1),
    Stroki(2, "барбер", 6, 2),
    Stroki(3, "ода", 3, 3),
    Stroki(4, "гильгамеш", 9, 3)
]

tables = [
    Table(1, "текст"),
    Table(2, "мало текста"),
    Table(3, "много текста")
]

TableStrings = [
    TableString(1, 1, 1),
    TableString(2, 2, 2),

```

```

        TableString(3, 3, 3),
        TableString(4, 4, 3),
        TableString(5,3,2) #Добавил эту строку сюда,чтобы показать связь
многие-ко-многим
]

```

```

print("1. Строки, начинаищиеся с 'б' и их таблицы:")
result1 = []
for stroka in StrokiList:
    if stroka.content.startswith('б'):
        table = next((t for t in tables if t.id == stroka.TableId), None)
        if table:
            result1.append((stroka.content, table.name))

```

```

for content, TableName in result1:
    print(f"  Строка: '{content}', Таблица: '{TableName}'")

```

```

print("\n2. Минимальное количество символов в строках таблиц
(сортировка по возрастанию):")

```

```

MinCharsInTable = {}
for table in tables:
    TableStrokiList = [s for s in StrokiList if s.TableId == table.id]
    if TableStrokiList:
        MinChars = min(s.CharAmount for s in TableStrokiList)
        MinCharsInTable[table.name] = MinChars

```

```

SortedMinChars = sorted(MinCharsInTable.items(), key=lambda x: x[1])
for TableName, MinChars in SortedMinChars:
    print(f"  Таблица '{TableName}': {MinChars} символов")

```

```

print("\n3. Все строки и их таблицы (сортировка по ID строки):")
result3 = []
for ts in TableStrings:
    stroka = next((s for s in StrokiList if s.id == ts.StringId), None)
    table = next((t for t in tables if t.id == ts.TableId), None)
    if stroka and table:
        result3.append((stroka.id, stroka.content, table.name))

```

```

result3_sorted = sorted(result3, key=lambda x: x[0])
for StringId, content, TableName in result3_sorted:
    print(f"  ID строки: {StringId}, Содержание: '{content}', Таблица:
'{TableName}'")

```

Код(RK2Tests.py)

```
from unittest import result
from RK2 import *
import pytest

stroki = [
    Stroki(1, "бла-бла", 7, 1),
    Stroki(2, "барбер", 6, 2),
    Stroki(3, "ода", 3, 3),
    Stroki(4, "гильгамеш", 9, 3)
]

tables = [
    Table(1, "текст"),
    Table(2, "мало текста"),
    Table(3, "много текста")
]

table_stroki = [
    TableStroka(1, 1, 1),
    TableStroka(2, 2, 2),
    TableStroka(3, 3, 3),
    TableStroka(4, 4, 3),
    TableStroka(5, 3, 2),
]

def TestRequestOne():
    result = RequestOne(stroki, tables)

    expected = [{ 'StringId': 1, 'StringContent': "бла-бла", 'TableName':
"текст", 'TableId': 1},
                { 'StringId': 2, 'StringContent': "барбер", 'TableName': "мало
текста", 'TableId': 2}]

    assert result == expected, f"Ожидали {expected}, получили {result}"

def TestRequestTwo():
    result = RequestTwo(stroki, tables)

    expected = [
        ("много текста", 3),
        ("мало текста", 6),
        ("текст", 7) ]

    assert result == expected, f"Ожидали {expected}, получили {result}"
```

```

def TestRequestThree():
    result = RequestThree(stroki, tables, table_stroki)

    expected = [
        {'StringId': 1, 'StringContent': 'бла-бла', 'TableName': 'текст',
        'TableId': 1},
        {'StringId': 2, 'StringContent': 'барбер', 'TableName': 'мало текста',
        'TableId': 2},
        {'StringId': 3, 'StringContent': 'ода', 'TableName': 'много
        текста', 'TableId': 3},
        {'StringId': 3, 'StringContent': 'ода', 'TableName': 'мало текста',
        'TableId': 2},
        {'StringId': 4, 'StringContent': 'гильгамеш', 'TableName': 'много
        текста', 'TableId': 3},]

    assert result == expected, f"Ожидали {expected}, получили {result}"

```

Работа кода:

```

===== test session starts =====
collecting ... collected 3 items

RK2Tests.py::testRequestOne PASSED [ 33%]
RK2Tests.py::testRequestTwo PASSED [ 66%]
RK2Tests.py::testRequestThree PASSED [100%]

===== 3 passed in 0.02s =====

```

```

(.venv) PS C:\Users\avefr\PycharmProjects\PythonLabs_3_semester\PK2> pytest -q RK2Tests.py
...
3 passed in 0.01s [100%]

```